

Capítulo 2

Núcleos de Procesamiento. Segmentación

2.1 Introducción: la arquitectura MIPS como caso de estudio

El procesador **MIPS** (*Microprocessor without Interlocked Pipeline Stages*) fue desarrollado por John Hennessy en la Universidad de Stanford entre 1981 y 1985, y se convirtió en el ejemplo canónico de arquitectura RISC (*Reduced Instruction Set Computer*). A diferencia de las arquitecturas CISC como el x86, el MIPS fue diseñado desde el principio para maximizar la eficiencia de la segmentación de cauce (*pipelining*): sus instrucciones tienen longitud fija (32 bits), acceden a memoria únicamente mediante instrucciones explícitas de carga (**lw**/**ld**) y almacenamiento (**sw**/**sd**), y emplean tres formatos ortogonales (R, I, J) que simplifican la decodificación hardware.

El ISA de 32 bits del MIPS define una “palabra” como 4 bytes, lo que implica que todas las instrucciones ocupan exactamente una palabra y que las instrucciones de memoria trabajan con múltiplos de 4 bytes (alineación). Esta uniformidad es precisamente lo que hace del MIPS un ejemplo pedagógico insuperable: los mecanismos de segmentación, los riesgos y las soluciones se pueden describir de forma limpia, sin los casos especiales que ensucian el análisis de arquitecturas más complejas. En la práctica, el MIPS estuvo presente en routers Cisco, equipos empotrados (TiVo), y consolas de videojuegos como la PlayStation, PSP y PS2.

2.1.1 Formatos de instrucción

Las instrucciones MIPS se codifican en tres formatos:

Cuadro 2.1: Formatos de instrucción MIPS (32 bits)

Tipo	Campos	Uso típico
R	op(6) rs(5) rt(5) rd(5) shamt(5) funct(6)	Operaciones ALU registro–registro
I	op(6) rs(5) rt(5) inmediato(16)	Cargas, almacenamientos, saltos condicionales
J	op(6) dirección(26)	Saltos incondicionales

Las instrucciones de tipo R se identifican por tener los 6 bits de mayor peso (*opcode*)

a cero; en ese caso, los 6 bits de menor peso (*funct*) codifican la operación aritmética concreta (*add*, *sub*, *and*, *or*, *slt*, etc.). En las instrucciones de tipo I, el campo de 16 bits se extiende en signo antes de usarse como desplazamiento en las operaciones de direccionamiento o como literal en las operaciones inmediatas.

➤ **Instrucciones aritméticas: tres registros como operandos.**

Instrucción	Ejemplo	Significado	Comentarios
suma	<code>add \$s1, \$s2, \$s3</code>	$\$s1 = \$s2 + \$s3$	Tres operandos, detecta overflow
resta	<code>sub \$s1, \$s2, \$s3</code>	$\$s1 = \$s2 - \$s3$	Tres operandos, detecta overflow
suma inmediata	<code>addi \$s1, \$s2, 100</code>	$\$s1 = \$s2 + 100$	Detecta overflow
suma sin signo.	<code>addu \$s1, \$s2, 100</code>	$\$s1 = \$s2 + \$s3$	Tres operandos, no detecta overflow
resta sin signo.	<code>subu \$s1, \$s2, 100</code>	$\$s1 = \$s2 - \$s3$	Tres operandos, no detecta overflow
multiplicación	<code>mul \$s1, \$s2</code>	$(Hi, Lo) = \$s1 \times \$s2$	Producto 64 bits en (Hi,Lo)
división	<code>div \$s1, \$s2</code>	$Lo = \$s1 / \$s2$ $Hi = \$s1 \text{ mod } \$s2$	Lo = división entera Hi = resto
Mover desde Hi	<code>mfhi \$s1</code>	$\$s1 = Hi$	Recupera valor de Hi
Mover desde Lo	<code>mflo \$s1</code>	$\$s1 = Lo$	Recupera valor de Lo

➤ **Instrucciones lógicas**

Instrucción	Ejemplo	Significado	Comentarios
and	<code>and \$s1, \$s2, \$s3</code>	$\$s1 = \$s2 \& \$s3$	Tres operandos
or	<code>or \$s1, \$s2, \$s3</code>	$\$s1 = \$s2 \$s3$	Tres operandos
and inmediato	<code>andi \$s1, \$s2, 100</code>	$\$s1 = \$s2 \& 100$	Tres operandos
or inmediato	<code>ori \$s1, \$s2, 100</code>	$\$s1 = \$s2 100$	Tres operandos
desplazamiento lógico izquierda	<code>sll \$s1, \$s2, 10</code>	$\$s1 = \$s2 \ll 10$	Despl. constante
desplazamiento lógico derecha	<code>srl \$s1, \$s2, 10</code>	$\$s1 = \$s2 \gg 10$	Despl. constante

➤ **Instrucciones de transferencia: direccionamiento base más desplazamiento. El primer operando no siempre es el destino**

Instrucción	Ejemplo	Significado	Comentarios
load word	<code>lw \$s1, 100(\$s2)</code>	$\$s1 = \text{Memory}[\$s2 + 100]$	Word de memoria a registro
store word	<code>sw \$s1, 100(\$s2)</code>	$\text{Memory}[\$s2 + 100] = \$s1$	Word de registro a memoria
load byte	<code>lb \$s1, 100(\$s2)</code>	$\$s1 = \text{Memory}[\$s2 + 100]$	Byte de memoria a registro
store byte	<code>sb \$s1, 100(\$s2)</code>	$\text{Memory}[\$s2 + 100] = \$s1$	Byte de registro a memoria
load upper immediate	<code>lui \$s1, 100</code>	$\$s1 = 100 * 2^{16}$	Carga sobre los 16 bits sup.

➤ **Instrucciones de salto condicionales o incondicionales que modifican el valor del registro contador de programa (PC)**

Instrucción	Ejemplo	Significado	Comentarios
branch on equal	<code>beq \$s1, \$s2, 25</code>	if ($\$s1 == \$s2$) goto $PC = PC + 4 + 100$	Relativo al PC
branch on not equal	<code>bne \$s1, \$s2, 25</code>	if ($\$s1 \neq \$s2$) goto $PC = PC + 4 + 100$	Relativo al PC
set on less than	<code>slt \$s1, \$s2, \$s3</code>	if ($\$s2 < \$s3$) $\$s1 = 1$ else $\$s1 = 0$	(instrucción aritmética)
set on less than imm.	<code>slti \$s1, \$s2, 100</code>	if ($\$s2 < 100$) $\$s1 = 1$ else $\$s1 = 0$	Complemento a dos
set on less than imm. unsigned	<code>sltiu \$s1, \$s2, 100</code>	if ($\$s2 < 100$) $\$s1 = 1$ else $\$s1 = 0$	Binario natural
jump	<code>j 2500</code>	$PC = 2500 * 4$	Absoluto
jump register	<code>j \$ra</code>	$PC = \$ra$	Absoluto
jump and link	<code>jal 2500</code>	$\$ra = PC$; $PC = 2500 * 4$	Absoluto

Figura 2.1: Diversas instrucciones utilizadas

2.2 Ciclo de instrucción y etapas de ejecución

Definición Ciclo de instrucción

El **ciclo de instrucción** es el conjunto de etapas que debe atravesar cualquier instrucción desde que se lee de memoria hasta que su resultado queda disponible para las instrucciones siguientes. En el MIPS segmentado con cinco etapas, estas son:

1. **IF** (*Instruction Fetch*): búsqueda de instrucción
2. **ID** (*Instruction Decode / Register Read*): decodificación y lectura de registros
3. **EX** (*Execute*): ejecución en la ALU o cálculo de dirección efectiva
4. **MEM** (*Memory Access*): acceso a la memoria de datos
5. **WB** (*Write Back*): escritura del resultado en el banco de registros

Cada etapa está implementada por un módulo hardware independiente y los resultados de una etapa se almacenan en **registros de segmentación** (*pipeline registers*) que separan etapas consecutivas: IF/ID, ID/EX, EX/MEM y MEM/WB. La función de estos registros es doble: propagan los operandos y las señales de control hacia las etapas posteriores, y permiten que distintas instrucciones coexistan simultáneamente en etapas distintas del cauce sin interferirse.

2.2.1 Descripción detallada de cada etapa

IF — Búsqueda de instrucción

Al comenzar cada ciclo de reloj, el hardware lee de la memoria de instrucciones (o caché L1 de instrucciones) la instrucción apuntada por el **Contador de Programa** (PC). Simultáneamente, se calcula la dirección de la siguiente instrucción secuencial sumando 4 al PC actual ($PC + 4$), y este valor se escribe en el registro PC para el siguiente ciclo. El valor de $PC + 4$ también se propaga al registro de segmentación IF/ID porque será necesario más adelante para calcular direcciones de salto (el destino de un salto condicional es $PC + 4 + \text{extensión_signo}(\text{inmediato} \times 4)$).

Es importante comprender que la etapa IF no interpreta el contenido de la instrucción: simplemente la lee y la propaga. No sabe si es una suma, una carga o un salto.

ID — Decodificación y lectura de registros

La etapa ID realiza varias operaciones en paralelo aprovechando el mismo ciclo de reloj:

- **Decodificación:** la lógica de control interpreta los bits del campo *opcode* (y *funct* para instrucciones tipo R) para generar las señales de control que gobernarán las etapas EX, MEM y WB. Estas señales se propagan a través de los registros de segmentación junto con los datos.

- **Lectura del banco de registros:** se leen simultáneamente los dos registros fuente (rs y rt). El banco de registros del MIPS tiene dos puertos de lectura y un puerto de escritura; los dos puertos de lectura se activan en paralelo, por lo que ambos registros fuente quedan disponibles al final de la etapa ID.
- **Extensión de signo:** el campo inmediato de 16 bits de las instrucciones tipo I se extiende a 32 bits preservando el signo. Este valor de 32 bits se usará en EX como segundo operando de la ALU (instrucciones inmediatas o accesos a memoria).
- **Cálculo de la dirección de salto potencial:** para minimizar la penalización por saltos, el MIPS añade en la etapa ID un sumador dedicado y un comparador de registros. El sumador calcula la dirección destino del salto ($PC + 4 + \text{extensión_signo}(\text{inmediato}) \times 4$). El comparador evalúa la condición del salto (igualdad de dos registros, en el caso de `beq/bne`). Si el salto se resuelve en ID, solo se pierde 1 ciclo en lugar de los 3 que se perderían si se resolviese en MEM.

EX — Ejecución

La etapa EX es el núcleo computacional del procesador. La ALU recibe sus dos operandos de entrada a través de multiplexores cuya selección está controlada por las señales generadas en ID:

- **Instrucciones aritméticas tipo R** (`add $rd, $rs, $rt`): la ALU suma (o resta, AND, OR, etc.) los valores de los registros rs y rt. El resultado se escribe en el registro de segmentación EX/MEM.
- **Instrucciones con inmediato tipo I** (`addi $rt, $rs, imm`): la ALU recibe el contenido del registro rs y el valor del inmediato extendido en signo. El multiplexor *ALUSrc* selecciona el inmediato en lugar del segundo registro.
- **Instrucciones de acceso a memoria** (`lw/sw`): la ALU calcula la dirección efectiva sumando el contenido del registro base y el desplazamiento (inmediato extendido en signo). Para una instrucción `sw`, el contenido del registro que se va a almacenar también pasa por esta etapa sin ser modificado, y se propaga al registro EX/MEM.
- **Instrucciones de salto:** aunque en el MIPS de 5 etapas los saltos se resuelven en ID (con hardware adicional), en una implementación sin ese hardware la ALU calcularía aquí la condición de salto. En el MIPS R4000 (pipeline de 8 etapas), la condición de salto y la dirección se calculan en EX.

MEM — Acceso a memoria

Solo las instrucciones de carga (`lw, ldc1`) y almacenamiento (`sw, sdc1`) realizan trabajo real en esta etapa. Para el resto de instrucciones, la etapa MEM es un ciclo de paso en el que los datos se propagan sin modificarse hacia el registro MEM/WB.

- **Instrucción `lw`:** la dirección calculada en EX se usa para leer la memoria de datos (caché L1 de datos). El dato leído se escribe en el registro MEM/WB.

- **Instrucción sw:** la dirección calculada en EX se usa para escribir en la memoria de datos el valor del registro *rt*, que fue leído en ID y propagado hasta aquí.
- **Instrucciones ALU:** el resultado de la ALU se propaga directamente a MEM/WB. La señal *MemRead* y *MemWrite* están ambas a cero.

WB — Escritura de resultado

En la etapa WB, el resultado de la instrucción (ya sea el valor leído de memoria para un *lw*, o el resultado de la ALU para instrucciones aritmético-lógicas) se escribe en el registro destino del banco de registros. Un multiplexor controlado por la señal *MemToReg* selecciona entre la salida de la memoria y la salida de la ALU.

¡Atención! Problema del número de registro erróneo en WB

En el diseño inicial del camino de datos segmentado, el número del registro destino está codificado en los bits 20–16 (campo *rt*) o 15–11 (campo *rd*) de la instrucción. Si no se propaga este número a través de los registros de segmentación, en la etapa WB se intentaría escribir en el registro equivocado (el que correspondía a la instrucción que estaba en WB 4 ciclos atrás). Por este motivo, el número del registro destino **debe propagarse explícitamente** a través de los registros IF/ID → ID/EX → EX/MEM → MEM/WB.

2.3 Implementaciones monociclo, multiciclo y segmentada

Para entender qué aporta la segmentación, es necesario comparar las tres implementaciones posibles del procesador MIPS.

2.3.1 Implementación monociclo

En la implementación **monociclo**, cada instrucción se ejecuta en un único ciclo de reloj. El CPI ideal es exactamente 1. Sin embargo, el período del reloj debe ser lo suficientemente largo para que la instrucción más lenta (típicamente la instrucción *lw*, que atraviesa las cinco etapas: lectura de instrucción, decodificación, cálculo de dirección, acceso a memoria y escritura en registro) pueda completarse en un solo ciclo.

Si una instrucción *add* tarda 600 ps y una instrucción *lw* tarda 800 ps, el período del reloj debe ser 800 ps. Las instrucciones *add* tendrán 200 ps de “tiempo muerto” en cada ciclo. Esta ineficiencia es el principal problema de la implementación monociclo.

2.3.2 Implementación multiciclo

En la implementación **multiciclo**, la ejecución de cada instrucción se divide en varias etapas, cada una de las cuales dura un ciclo de reloj más corto. Una instrucción *add* podría completarse en 4 ciclos de reloj cortos, y una *lw* en 5 ciclos. Como el ciclo de reloj es más corto (equivale a la etapa más lenta, no a la instrucción más lenta completa), se aprovecha mejor el hardware.

El problema de la implementación multiciclo sin segmentación es que solo hay una instrucción en ejecución en cada momento. Aunque el ciclo de reloj es más corto, la siguiente instrucción solo puede comenzar cuando la anterior haya terminado completamente.

2.3.3 Implementación segmentada

La **implementación segmentada** (pipeline) extiende la multiciclo permitiendo que varias instrucciones se ejecuten simultáneamente, cada una en una etapa diferente. La analogía con la cadena de montaje de una fábrica es precisa: en una cadena de montaje, el coche que entra en la línea no espera a que el primer coche haya salido para comenzar su ensamblaje; simplemente avanza etapa por etapa, solapándose con los coches anteriores.

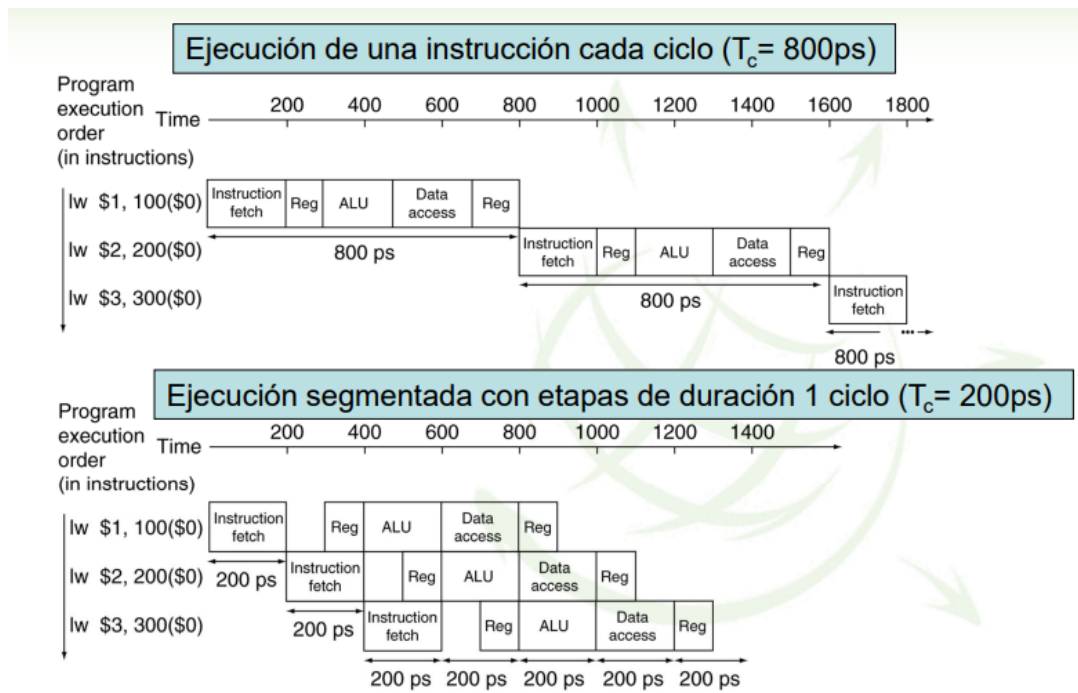


Figura 2.2: Comparación entre implementación monociclo (800 ps/ciclo, $\text{CPI}=1$) e implementación segmentada (200 ps/ciclo, $\text{CPI}\approx 1$ ideal). La segmentación reduce el tiempo entre instrucciones completadas (*throughput*) pero no el tiempo de ejecución de una instrucción individual (*latencia*).

2.3.4 Tiempo del pipeline y aceleración

En una unidad **no segmentada** con N etapas de tiempos $T_1, \dots, T_N$, el tiempo para ejecutar una instrucción es:

$$T_{\text{comb}} = T_1 + T_2 + \dots + T_N$$

En la unidad **segmentada**, el período del reloj debe acomodar la etapa más lenta más el tiempo de carga del registro de segmentación T_R :

$$T_{\text{clk}} = \max(T_R + T_1, T_R + T_2, \dots, T_R + T_N)$$

El tiempo para ejecutar k instrucciones en la versión segmentada (tras la latencia inicial) es:

$$T_{\text{seg}}(k) = N \cdot T_{\text{clk}} + (k - 1) \cdot T_{\text{clk}} = (N + k - 1) \cdot T_{\text{clk}}$$

La aceleración ideal de la versión segmentada frente a la monociclo para k instrucciones es:

$$S = \frac{k \cdot T_{\text{comb}}}{(N + k - 1) \cdot T_{\text{clk}}} \xrightarrow{k \rightarrow \infty} \frac{T_{\text{comb}}}{T_{\text{clk}}} = \frac{T_1 + T_2 + \dots + T_N}{\max(T_R + T_i)}$$

Concepto clave La segmentación mejora el throughput, no la latencia

La segmentación **no** reduce el tiempo que tarda en ejecutarse una instrucción individual (latencia). Una instrucción **lw** sigue tardando 5 ciclos de reloj en el procesador segmentado, igual que en el multiciclo. Lo que mejora es la **productividad** (*throughput*): el número de instrucciones completadas por unidad de tiempo. En estado estacionario, idealmente se completa una instrucción por ciclo de reloj.

Ejemplo Cálculo de aceleración por segmentación

Un procesador no segmentado tiene:

- Período de reloj: 1 ns/ciclo
- 40% instrucciones ALU: 4 ciclos cada una
- 20% instrucciones de bifurcación: 4 ciclos cada una
- 40% instrucciones de memoria: 5 ciclos cada una
- Sobrecoste de segmentación: 0,2 ns por ciclo

Tiempo medio por instrucción en el original:

$$\begin{aligned} t_{\text{orig}} &= 1 \text{ ns/ciclo} \times (0,6 \times 4 + 0,4 \times 5) \text{ ciclos/instrucción} \\ &= 1 \times 4,4 = 4,4 \text{ ns/instrucción} \end{aligned}$$

Tiempo medio por instrucción con segmentación (CPI ideal = 1):

$$t_{\text{seg}} = (1 + 0,2) \text{ ns/ciclo} \times 1 \text{ ciclo/instrucción} = 1,2 \text{ ns/instrucción}$$

Aceleración:

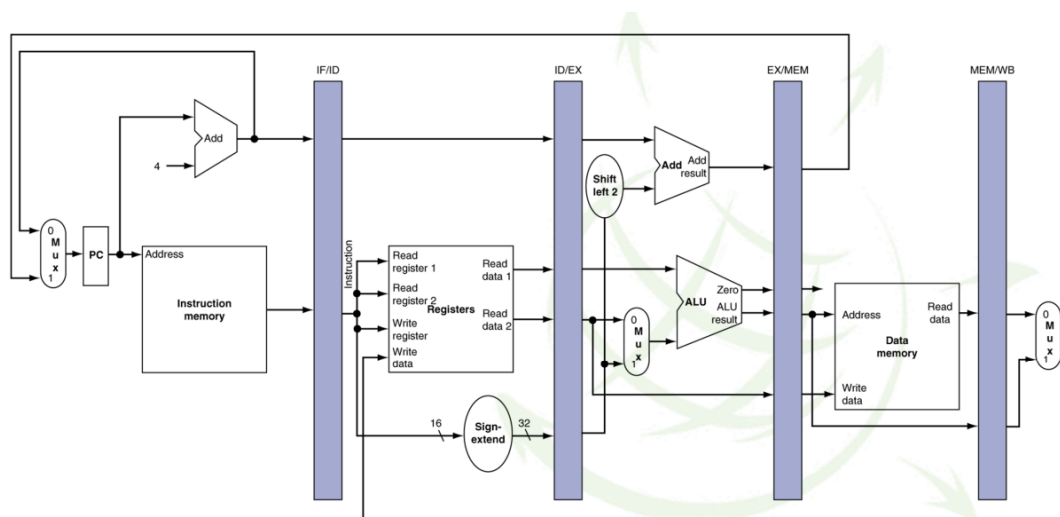
$$S = \frac{4,4}{1,2} \approx 3,7$$

2.4 Señales de control en el pipeline

Cada instrucción propaga un conjunto de señales de control a través de los registros de segmentación. Estas señales gobiernan el comportamiento de los módulos en las etapas EX, MEM y WB. La tabla 2.2 resume las señales más importantes.

Cuadro 2.2: Señales de control del pipeline MIPS de 5 etapas

Señal	Etapla activa	Función
RegWrite	WB	1 = escribir resultado en banco de registros
MemToReg	WB	1 = escribir dato de memoria; 0 = resultado ALU
MemRead	MEM	1 = leer dato de memoria (instrucción lw)
MemWrite	MEM	1 = escribir dato en memoria (sw)
Branch	MEM/ID	1 = instrucción de salto condicional
ALUOp	EX	2 bits que seleccionan la operación de la ALU
ALUSrc	EX	1 = usar inmediato; 0 = usar registro <i>rt</i>
RegDst	EX	1 = destino es campo <i>rd</i> ; 0 = destino es campo <i>rt</i>



Cuadro 2.3: Valores de las señales de control para los tipos de instrucción más comunes

Instrucción	RegDst	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
add (R-type)	1	0	0	1	0	0	0	10
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00
beq	X	0	X	0	0	0	1	01

X = valor irrelevante (no afecta al resultado)

2.5 Riesgos en la ejecución (*hazards*)

Definición Riesgo (hazard)

Un **riesgo** es cualquier situación que impide que una instrucción pueda comenzar su ejecución en el ciclo de reloj previsto, rompiendo el ideal de emitir una instrucción por ciclo. Los riesgos reducen el CPI efectivo por encima del valor ideal de 1. Los ciclos en los que no se completa ninguna instrucción útil se denominan **burbuja** (*stalls* o NOPs) del pipeline.

Existen tres tipos de riesgos, que se analizan en detalle a continuación.

2.5.1 Riesgos estructurales

Los **riesgos estructurales** se producen cuando dos instrucciones simultáneamente en el pipeline necesitan usar el *mismo recurso hardware* en el mismo ciclo de reloj.

Ejemplo Riesgo estructural por memoria unificada

Si el MIPS dispusiera de una única memoria para instrucciones y datos (en lugar de cachés separadas), la instrucción en etapa IF necesitaría leer una instrucción mientras la instrucción en etapa MEM necesita leer o escribir un dato. Ambas acceden a la misma memoria en el mismo ciclo: riesgo estructural.

La solución adoptada en el MIPS (y en todos los procesadores modernos) es disponer de **cachés separadas** para instrucciones y datos (cache split L1-I y L1-D), eliminando este riesgo estructural.

Ejemplo Riesgo estructural por unidad de división no segmentada

La unidad de división en punto flotante del MIPS no está segmentada: mientras se ejecuta una división, ninguna otra división puede comenzar. Si en el código aparecen dos instrucciones `div.d` consecutivas, la segunda debe esperar a que la primera complete su etapa EX (que dura múltiples ciclos). Esto es un riesgo estructural.

A diferencia de la memoria, no se puede “duplicar” fácilmente una unidad de división, pues su coste en área es muy alto. La solución es detectar el riesgo en la etapa ID e insertar las paradas necesarias.

Ejemplo Riesgo estructural por conflicto en escritura (WB)

En procesadores con operaciones multiciclo en punto flotante, varias instrucciones pueden alcanzar la etapa WB en el mismo ciclo (por ejemplo, una suma en FP que tarda 4 ciclos y una multiplicación que tarda 7 pueden terminar simultáneamente). Si el banco de registros solo tiene un puerto de escritura, habrá un riesgo estructural. La solución es detener la emisión de instrucciones cuando se detecta que más instrucciones que puertos de escritura llegarán a WB en el mismo ciclo.

Para comparar diseños con y sin riesgos estructurales:

Ejemplo Comparación de diseños: con y sin riesgos estructurales

- **Diseño A** (sin riesgos estructurales): $T_A = 1$ ns, CPI = 1.
- **Diseño B** (con riesgos estructurales): $T_B = 0,9$ ns, CPI = 1 para el 70% de instrucciones, CPI = 2 para el 30% restante (accesos a datos con conflicto de 1 ciclo).

$$t_{\text{inst}}(A) = 1 \times 1 \text{ ns} = 1 \text{ ns}$$

$$t_{\text{inst}}(B) = (0,7 \times 1 + 0,3 \times 2) \times 0,9 \text{ ns} = 1,3 \times 0,9 = 1,17 \text{ ns}$$

El diseño A es más rápido por instrucción a pesar de tener mayor período de reloj.

2.5.2 Riesgos de datos

Los **riesgos de datos** se producen cuando el orden de acceso a los operandos es modificado por la ejecución solapada del pipeline. Se clasifican en tres tipos:

RAW — Read After Write (dependencia verdadera)

Definición Riesgo RAW

Un riesgo **RAW** (*Read After Write*) ocurre cuando una instrucción j intenta leer un registro que todavía no ha sido escrito por una instrucción anterior i . Es la **dependencia verdadera**: el valor que j necesita es el que i produce, y ese valor aún no está disponible cuando j lo necesita. Es el único tipo de riesgo de datos que puede ocurrir en un pipeline de 5 etapas tipo MIPS.

Ejemplo Riesgo RAW sin forwarding

```
1  add $1, $2, $3    # escribe $1 en WB (ciclo 5)
2  sub $4, $1, $3    # lee $1 en ID (ciclo 3): RIESGO RAW en $1
```

Listing 2.1: Riesgo RAW entre instrucciones consecutivas

Cuando **sub** está en la etapa ID (ciclo 3), **add** está en la etapa EX (ciclo 3): el resultado de **add** no estará en el banco de registros hasta el ciclo 5 (WB). Sin forwarding, **sub** debe esperar **2 ciclos** (paradas) antes de poder leer el valor correcto de **\$1**.

WAR — Write After Read (antidependencia)

Definición Riesgo WAR

Un riesgo **WAR** (*Write After Read*) ocurre cuando una instrucción j intenta escribir un registro antes de que una instrucción anterior i lo haya leído. Es conocido como *antidependencia* en el contexto de compiladores.

¡Atención! WAR no puede ocurrir en MIPS de 5 etapas

En el MIPS de 5 etapas, las lecturas de registros siempre ocurren en la etapa **ID** (etapa 2) y las escrituras siempre ocurren en la etapa **WB** (etapa 5). Dado que en el pipeline las instrucciones avanzan en orden, cuando la instrucción j está en WB (etapa 5), la instrucción i ya ha pasado por ID (etapa 2) hace 3 ciclos. Por tanto, la escritura siempre ocurre *después* de la lectura: los riesgos WAR **no pueden producirse** en este pipeline.

WAW — Write After Write (dependencia de salida)

Definición Riesgo WAW

Un riesgo **WAW** (*Write After Write*) ocurre cuando una instrucción j intenta escribir un registro antes de que una instrucción anterior i complete su escritura en ese mismo registro.

¡Atención! WAW no puede ocurrir en MIPS de 5 etapas (enteros)

En el MIPS de 5 etapas para operaciones enteras, todas las instrucciones siguen estrictamente el mismo orden y tardan el mismo número de ciclos, por lo que la instrucción i siempre llega a WB antes que j . Los riesgos WAW **no pueden producirse** en este caso.

Sin embargo, en procesadores con operaciones multiciclo en punto flotante (donde distintas instrucciones pueden tener latencias de EX diferentes: 4 ciclos para suma FP, 7 para multiplicación FP), una instrucción posterior podría completar WB antes que una anterior, generando riesgos WAW. Estos se detectan en la etapa ID comprobando si alguna instrucción en etapas posteriores tiene el mismo registro destino pero llegará a WB más tarde.

Cuadro 2.4: Resumen de riesgos de datos y su ocurrencia en MIPS de 5 etapas

Tipo	Descripción	Nombre en compiladores	¿Ocurre en MIPS?	Solución
RAW	j lee antes de que i escriba	Dependencia verdadera	Sí	Forwarding, bloqueo
WAR	j escribe antes de que i lea	Antidependencia	No	Renombrado de registros
WAW	j escribe antes de que i escriba	Dep. de salida	No (sí en FP)	Renombrado / detención

2.6 Soluciones a los riesgos RAW

2.6.1 Bloqueo (*stall*): la solución más simple

La solución más simple ante un riesgo RAW consiste en **detener** la instrucción dependiente en la etapa ID hasta que el dato esté disponible en el banco de registros. Durante el bloqueo se insertan **burbujas** (NOPs) en la etapa EX para que el pipeline avance sin hacer trabajo útil.

El número de ciclos de bloqueo depende del tipo de instrucción productora: en el MIPS básico (sin forwarding), una instrucción ALU produce su resultado al final de EX (ciclo 3 desde IF), pero no lo escribe en el banco de registros hasta WB (ciclo 5). Por tanto, la instrucción dependiente que está en ID en el mismo ciclo que la productora en EX necesita esperar **2 ciclos**.

¡Atención! Regla de bloqueo sin forwarding

Sin hardware de forwarding, la instrucción dependiente debe esperar en la etapa ID hasta que la instrucción productora complete su etapa WB. El MIPS permite que un registro sea escrito en la **primera mitad** de un ciclo y leído en la **segunda mitad** del mismo ciclo (*register file with write-then-read in same cycle*), lo que reduce en 1 el número de paradas necesarias.

2.6.2 Reordenamiento de instrucciones por el compilador

El compilador puede reordenar el código para *separar* la instrucción productora de la dependiente intercalando instrucciones independientes. Si no existen instrucciones independientes disponibles, se insertan NOPs explícitos.

Ejemplo Reordenamiento de código para eliminar riesgos RAW

```
1 lw $t1, 0($t0)    # I1: carga A[i]
2 lw $t2, 4($t0)    # I2: carga B[i]
3 add $t3, $t1, $t2  # I3: depende de I1 e I2 -> stall x 2
4 sw $t3, 12($t0)   # I4: depende de I3 -> stall x 1 (con fw)
5 lw $t4, 8($t0)    # I5: carga C[i]
6 add $t5, $t1, $t4  # I6: depende de I1, I5 -> stall
7 sw $t5, 16($t0)   # I7: depende de I6 -> stall
```

Listing 2.2: Código original con riesgos RAW (23 ciclos)

```
1 lw $t1, 0($t0)    # I1: carga A[i]
2 lw $t2, 4($t0)    # I2: carga B[i]
3 lw $t4, 8($t0)    # I5 adelantada: rellena hueco de I1 e I2
4 add $t3, $t1, $t2  # I3: $t1 y $t2 ya disponibles
5 sw $t3, 12($t0)   # I4: stall 1 ciclo
6 add $t5, $t1, $t4  # I6: $t4 ya disponible
7 sw $t5, 16($t0)   # I7
```

Listing 2.3: Código reordenado (19 ciclos, sin forwarding)

2.6.3 Forwarding (adelantamiento o bypassing)

Definición Forwarding

El **forwarding** (también llamado *adelantamiento*, *anticipación* o *bypassing*) es una técnica hardware que permite proporcionar a las entradas de la ALU resultados calculados por instrucciones anteriores que aún no han completado la etapa WB, tomándolos directamente de los registros de segmentación donde ya están disponibles.

La idea fundamental es que cuando la instrucción productora completa su etapa EX, el resultado queda almacenado en el registro de segmentación EX/MEM. Este valor puede “enviarse hacia atrás” (hacia la etapa EX de la instrucción dependiente) mediante rutas adicionales (*bypass paths*) y multiplexores. No hace falta esperar a que el resultado se escriba en el banco de registros.

Los casos de forwarding en el MIPS de 5 etapas son:

1. **Forwarding EX → EX:** la instrucción dependiente está en EX y la productora acaba de salir de EX (está en MEM). El resultado se toma directamente del registro EX/MEM.
2. **Forwarding MEM → EX:** la instrucción dependiente está en EX y la productora está en WB (acaba de salir de MEM). El resultado se toma del registro MEM/WB.

La lógica de forwarding se implementa comparando los campos rs y rt de la instrucción en EX con los campos rd de las instrucciones en MEM y WB:

```
1  # Forwarding desde etapa MEM (EX/MEM) hacia entrada A de ALU:
2  if (EX/MEM.RegWrite AND EX/MEM.RegisterRd != 0
3      AND EX/MEM.RegisterRd == ID/EX.RegisterRs)
4      ForwardA = 10  # usar salida de EX/MEM
5
6  # Forwarding desde etapa WB (MEM/WB) hacia entrada A de ALU:
7  if (MEM/WB.RegWrite AND MEM/WB.RegisterRd != 0
8      AND MEM/WB.RegisterRd == ID/EX.RegisterRs
9      AND NOT (EX/MEM.RegWrite AND EX/MEM.RegisterRd != 0
10             AND EX/MEM.RegisterRd == ID/EX.RegisterRs))
11      ForwardA = 01  # usar salida de MEM/WB
```

Listing 2.4: Condiciones de forwarding (pseudocódigo)

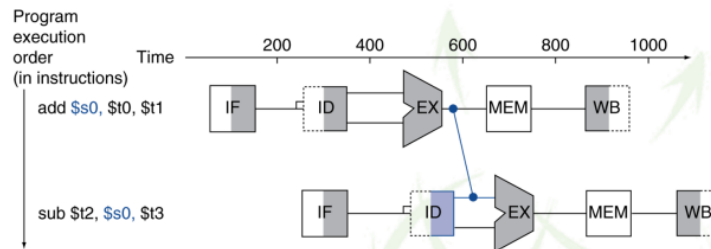
La condición “AND NOT ...” de la segunda regla garantiza que si tanto EX/MEM como MEM/WB podrían hacer forwarding del mismo registro, se usa el valor más reciente (EX/MEM).

Límite del forwarding: el riesgo load-use

El forwarding no puede resolver todos los riesgos RAW. En particular, el **riesgo load-use** es inevitable con forwarding:

```
1 lw $1, 45($2)  # carga de memoria: resultado disponible al
                  FINAL de MEM
```

➤ **Forwarding** (también llamado adelantamiento o anticipación o bypassing): el hardware adelanta el dato que hace falta una vez éste está calculado, sin necesidad de esperar a la escritura en el registro. Requiere conexiones adicionales. Soluciona dependencias tipo RAW.



➤ **Bloqueo junto con forwarding:** parar por hardware la ejecución de las instrucciones en el cauce esperando a que el dato necesario esté disponible. Si fuese imposible realizar *forwarding* el bloqueo duraría un ciclo más, como se ve en este ejemplo.

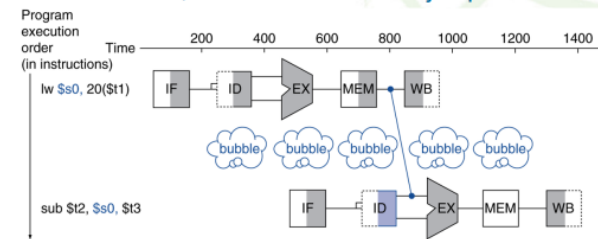


Figura 2.3: Camino de datos segmentado del MIPS con rutas de forwarding (adelantamiento). Las flechas en rojo muestran las rutas bypass desde EX/MEM y MEM/WB hacia las entradas de la ALU.

2 `add $5, $1, $7 # necesita $1 al INICIO de EX: 1 ciclo de adelanto imposible`

Listing 2.5: Riesgo load-use: un stall inevitable

Cuando `add` está en EX (ciclo 4), la instrucción `lw` está en MEM (ciclo 4). El dato leído de memoria por `lw` no estará disponible hasta el *final* del ciclo 4, pero `add` lo necesita al *principio* del ciclo 4. Es físicamente imposible hacer el forwarding: el dato no existe todavía. Se debe insertar **1 ciclo de parada**.

2.6.4 Control de interbloqueo de instrucciones (hazard detection unit)

La **unidad de detección de conflictos** (*hazard detection unit*) detecta automáticamente en hardware la necesidad de insertar paradas. Se activa cuando se cumplen simultáneamente:

1. La etapa ID contiene una instrucción de tipo ALU, salto o almacenamiento.
2. La etapa EX está ejecutando una instrucción de carga (`lw`/`ld`).
3. El registro destino de la instrucción de carga coincide con algún operando fuente de la instrucción en ID.

Cuando se activa la detección de interbloqueo:

- Se congela la instrucción en ID (y todas las anteriores en IF) durante 1 ciclo.

- Se inserta una burbuja (NOP) en la etapa EX.
- El PC no avanza durante ese ciclo.

```

1 LW    R1, 45(R2)    # EX (ciclo 3): carga en R1
2 ADD   R5, R1, R7    # ID (ciclo 3): necesita R1 -> STALL 1 ciclo
3 SUB   R8, R6, R7    # IF (ciclo 3): congelada
4 OR    R9, R6, R7    # esperando en memoria

```

Listing 2.6: Ejemplo de detección de interbloqueo load-use

2.7 Riesgos de control: instrucciones de salto

Definición Riesgo de control

Un **riesgo de control** se produce cuando el pipeline no puede conocer la dirección de la siguiente instrucción que debe ejecutar porque depende del resultado de una instrucción de salto que todavía se está ejecutando.

En el MIPS básico, si la condición de salto se evaluase en la etapa MEM (etapa 4), en el momento en que el salto resuelve su destino, ya se habrían captado y decodificado las 3 instrucciones siguientes. Si el salto se toma, esas 3 instrucciones son incorrectas y deben descartarse (flush), perdiendo 3 ciclos. Para mitigar este coste, el MIPS añade hardware en la etapa ID:

- Un **comparador** de registros que evalúa la condición de salto (igualdad) en ID.
- Un **sumador** que calcula la dirección destino del salto en ID.

Con este hardware, el salto se resuelve en la etapa ID, reduciendo la penalización a solo **1 ciclo** en lugar de 3.

2.7.1 Primera opción: congelación del pipeline

La solución más conservadora consiste en **detener el pipeline** en cuanto se detecta una instrucción de salto, esperando hasta que la condición y la dirección destino estén disponibles antes de captar la siguiente instrucción. En el MIPS con resolución en ID, esto supone exactamente 1 ciclo de penalización por cada salto.

El coste total de los saltos con congelación es:

$$\text{CPI}_{\text{efectivo}} = 1 + f_{\text{saltos}} \times p_{\text{penalización}}$$

donde f_{saltos} es la fracción de instrucciones que son saltos y $p_{\text{penalización}}$ es el número de ciclos perdidos por salto.

2.7.2 Segunda opción: predicción fija

Predicción de salto no tomado

El procesador asume que el salto *no* se toma y continúa captando instrucciones secuenciales. Si la predicción es correcta (el salto no se toma), no hay penalización. Si la predicción es incorrecta (el salto sí se toma), las instrucciones captadas tras el salto se descartan (se convierten en NOPs) y se captan las instrucciones en el destino del salto. En el MIPS de 5 etapas con resolución en ID, esto supone 1 ciclo de penalización por predicción incorrecta.

Predicción de salto tomado

El procesador asume que el salto *siempre* se toma y comienza a captar instrucciones desde la dirección destino tan pronto como se calcula. Sin embargo, en un pipeline de 5 etapas donde la dirección destino se calcula en ID, no se gana ninguna ventaja respecto a la congelación: cuando se conoce la dirección, ya se ha perdido 1 ciclo de todas formas. La predicción de salto tomado es útil en procesadores más profundos donde hay más ciclos entre el inicio de la búsqueda y el cálculo de la dirección destino.

¡Atención! En el MIPS de 5 etapas, la predicción de salto no tomado es la correcta

Dado que en el MIPS de 5 etapas tanto la predicción tomado como la no tomado resultan en 1 ciclo de penalización cuando la predicción falla, la diferencia radica en la frecuencia de fallo. La predicción “no tomado” es óptima cuando los saltos hacia atrás (bucles) se toman con más frecuencia que los saltos hacia adelante (condiciones), lo que se estima en la mayoría de programas reales.

2.7.3 Tercera opción: bifurcación retardada (*delayed branch*)

Definición Bifurcación retardada / Delayed Branch

En la **bifurcación retardada**, el ISA define que la instrucción inmediatamente posterior al salto (la **ranura de retardo** o *delay slot*) se ejecuta *siempre*, independientemente de si el salto se toma o no. El compilador es responsable de rellenar la ranura con una instrucción útil.

En el pipeline del MIPS de 5 etapas con resolución del salto en la etapa ID, hay exactamente **1 ranura de retardo**. Esto significa que la instrucción que sigue al salto en el código binario se ejecuta siempre antes de que el control llegue al destino del salto (si se toma) o a la instrucción tras la ranura (si no se toma).

Hay tres estrategias para rellenar la ranura, por orden de preferencia:

1. **(a) Mover una instrucción anterior al salto:** se toma una instrucción que precede al salto y que sea independiente de la condición de salto y del resultado del salto. Esta es la estrategia óptima porque no duplica código y la instrucción siempre produce un resultado útil.
2. **(b) Copiar la instrucción de destino del salto:** se duplica la primera instrucción del bloque destino. Produce una instrucción extra pero rellena la

ranura. Solo es válida si la instrucción puede ejecutarse sin efectos secundarios cuando el salto no se toma.

3. (c) **Mover una instrucción que sigue al salto:** se toma una instrucción del flujo de no-salto. Solo es válida si es inocua cuando el salto se toma (no cambia el estado del sistema).

¡Atención!

La instrucción en la ranura de retardo **no puede** modificar ningún registro o posición de memoria que sea relevante para la condición de salto, pues en el momento en que se ejecuta aún no se sabe si el salto se tomará o no. La ranura debe rellenarse con una instrucción que sea correcta *en cualquier caso*.

2.7.4 Cuarta opción: predicción dinámica de saltos

En procesadores con pipelines más profundos que el MIPS de 5 etapas (y en los procesadores superescalares del Tema 3), la penalización por salto puede ser de 10 o más ciclos, haciendo que las técnicas anteriores sean insuficientes. La solución moderna es la **predicción dinámica**: el hardware mide el comportamiento real de cada salto y adapta sus predicciones.

Predictor de 1 bit

Se asocia **1 bit de historia** a cada instrucción de salto (indexada por los bits menos significativos de su dirección). El bit almacena el resultado de la última ejecución de ese salto:

- 1 = el salto fue tomado → predecir tomado para la próxima vez.
- 0 = el salto no fue tomado → predecir no tomado.

Si la predicción falla, el bit se complementa. El predictor de 1 bit tiene un problema en bucles anidados: en la última iteración del bucle interno, el salto no se toma, cambia la predicción a “no tomado”. Al inicio de la siguiente iteración del bucle externo, el salto interno se toma pero se predice “no tomado”: fallo. Esto genera 2 fallos por iteración del bucle externo.

Predictor de 2 bits

El predictor de 2 bits usa una **máquina de estados de 4 estados** para añadir “inercia” a la predicción: se necesitan dos predicciones consecutivas incorrectas para cambiar la predicción. Los cuatro estados son:

- **11** (Fuertemente tomado, FT): predice tomado; si falla → 10
- **10** (Débilmente tomado, DT): predice tomado; si falla → 01
- **01** (Débilmente no tomado, DNT): predice no tomado; si falla → 10
- **00** (Fuertemente no tomado, FNT): predice no tomado; si falla → 01

El predictor de 2 bits elimina el 50% de los fallos del de 1 bit para secuencias de saltos tipo T–NT–T–NT–T–NT (alternadas).

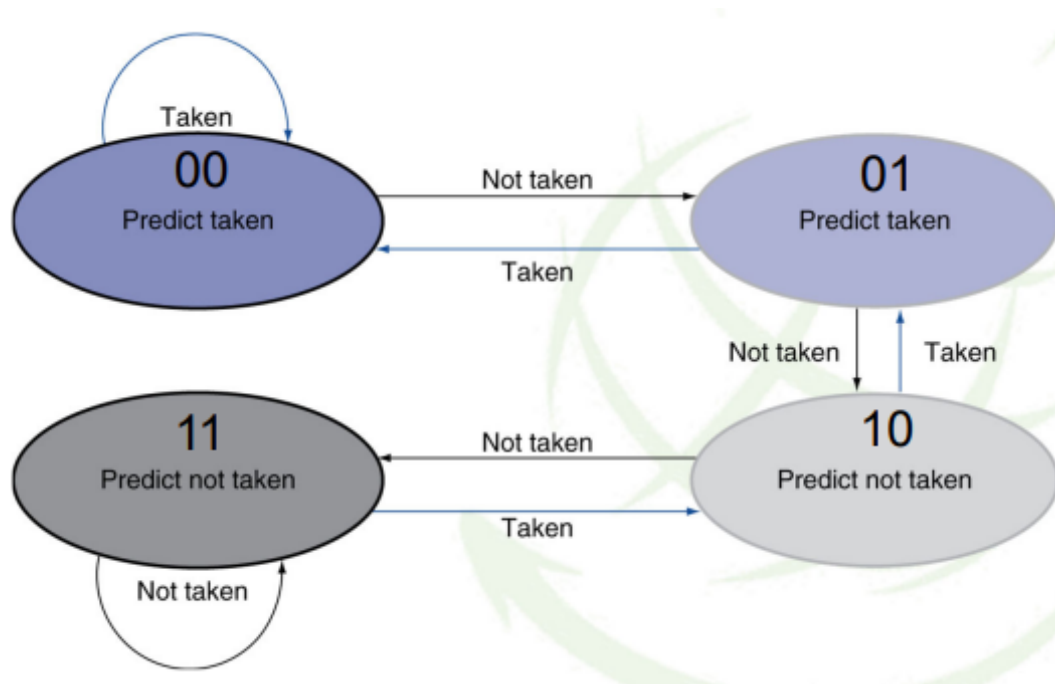


Figura 2.4: Máquina de estados del predictor dinámico de 2 bits. Los estados marcados como “tomado” son 11 y 10; los marcados como “no tomado” son 01 y 00. Solo dos predicciones incorrectas consecutivas cambian entre los grupos.

Predictor basado en información local

El predictor de información local va más allá de registrar el *último* resultado de un salto: almacena los resultados de las **últimas k ocurrencias** del mismo salto. Con estos k bits de historia se indexa una memoria que contiene los bits de estado del predictor. La ventaja es que puede predecir perfectamente patrones periódicos del tipo T–NT–T–NT: si $k \geq 2$, el predictor aprende que tras “T” siempre viene “NT” y viceversa.

Predictor basado en información global

El predictor de información global utiliza el resultado de las **últimas N instrucciones de salto** (de cualquier salto, no solo del salto actual) para indexar la memoria de predicción. Se implementa con un registro de desplazamiento (*global history register*) que almacena el resultado (T/NT) de los últimos N saltos. El contenido del registro indexa una tabla de 2 bits de estado.

Predictor combinado

Los procesadores reales combinan un predictor local y uno global, seleccionando dinámicamente cuál usar para cada salto individual. La selección se basa en un contador de saturación de 2 bits que registra cuál de los dos predictores ha acertado más recientemente para ese salto. La tasa de acierto de los saltos en procesadores reales depende del tipo de programa y suele estar entre el 85% y el 99%.

Buffer de destino de salto (*Branch Target Buffer, BTB*)

Incluso con un predictor dinámico de alta tasa de acierto, se pierde un ciclo calculando la dirección destino del salto. La solución es el **BTB**: una caché indexada por el PC de la instrucción de salto que almacena la dirección destino calculada en ejecuciones anteriores. Si el BTB acierta y el predictor dice “tomado”, la siguiente instrucción puede captarse en la dirección destino ya en el ciclo IF, sin penalización alguna.

2.8 Riesgos de datos y de control: interacciones

Cuando la condición de un salto depende del resultado de una instrucción aritmético-lógica inmediatamente anterior, se produce un riesgo de datos que interactúa con el riesgo de control. La resolución del salto en la etapa ID del MIPS introduce dependencias adicionales:

- Si el registro de comparación es el destino de una instrucción ALU que está en EX (2 instrucciones antes del salto), se puede resolver usando **forwarding desde EX/MEM hacia el comparador de ID**: 0 ciclos de penalización adicional.
- Si el registro de comparación es el destino de una instrucción ALU que está en ID (1 instrucción antes del salto), se necesita **1 ciclo de bloqueo** adicional. El forwarding llega desde EX (donde la ALU todavía está calculando el resultado cuando el salto llega a ID).
- Si el registro de comparación es el destino de una instrucción **lw** inmediatamente anterior al salto, se necesitan **2 ciclos de bloqueo** adicionales.

2.9 Extensión a operaciones multiciclo: punto flotante

Las instrucciones de punto flotante (FP) presentan características distintas que complican el pipeline:

2.9.1 Unidades funcionales especializadas

El MIPS con soporte de punto flotante añade las siguientes unidades funcionales en la etapa EX, cada una con su propia latencia:

- **Unidad de suma FP**: 4 etapas de ejecución (latencia de 4 ciclos adicionales si hay dependencia RAW con la instrucción siguiente).
- **Unidad de multiplicación FP**: 7 etapas (latencia adicional de 6 ciclos).
- **Unidad de división FP**: *no segmentada*. Una nueva división solo puede comenzar cuando la anterior ha terminado completamente. Esto genera riesgos estructurales que deben detectarse en ID.

Cuadro 2.5: Latencias adicionales de instrucciones en el MIPS con FP (ciclos de espera adicionales cuando hay dependencia RAW)

Instrucción productora	Instrucción consumidora	Ciclos de espera adicionales
ldc1 (carga FP doble)	Instr. FP	2
add.d (suma FP)	Instr. FP	4
mul.d (mult. FP)	Instr. FP	6
div.d (div. FP)	Instr. FP	variable (no segmentada)
addi/subi (enteros)	Instr. enteros	0 (con forwarding)

2.9.2 Riesgos adicionales en el pipeline FP

La presencia de unidades multiciclo introduce dos tipos adicionales de riesgos que no existen en el pipeline de enteros de 5 etapas:

1. **Riesgos WAW en FP:** si se emiten dos instrucciones que escriben en el mismo registro FP pero con latencias distintas, la segunda podría completar WB antes que la primera. La unidad de detección compara el registro destino de la instrucción en ID con los registros destino de todas las instrucciones en etapas posteriores; si hay coincidencia con una instrucción que llegaría a WB *después*, se inserta una parada.
2. **Riesgo estructural en WB:** si varias instrucciones FP de distinta latencia se emiten en ciclos consecutivos, pueden llegar a WB en el mismo ciclo. Si el banco de registros FP solo tiene un puerto de escritura, hay un riesgo estructural. La solución es detectar en ID que se produciría esta situación e insertar las paradas necesarias.

2.10 Excepciones en el pipeline

Definición Excepción

Una **excepción** es un evento que altera el flujo normal de ejecución del programa, generado por un evento interno a la CPU (código de operación no definido, desbordamiento aritmético, syscall) o externo (interrupción hardware). Las excepciones requieren que el procesador guarde el estado actual, salte a una rutina de tratamiento, y posteriormente (en algunos casos) reanude la ejecución desde el punto de la excepción.

Definición Excepción precisa

Una **excepción precisa** es aquella que permite recuperar el estado del procesador exactamente como estaba en el momento de la instrucción que causó la excepción, como si ninguna instrucción posterior a ella hubiera comenzado a ejecutarse.

Las excepciones precisas son difíciles de implementar en procesadores con pipelines profundos y unidades multiciclo, porque cuando una instrucción genera una excepción en la etapa EX, instrucciones posteriores ya pueden haber modificado el estado del sistema (escrito registros, escrito memoria). Hay varias estrategias para gestionar este problema:

1. **Banco de registros adicional (shadow registers):** se guarda una copia de los valores de los registros antes de cada escritura. En caso de excepción, se restauran los valores anteriores.
2. **Rutina software de repetición:** se utiliza una rutina del sistema operativo que repite la ejecución de las instrucciones desde la que causó la excepción, deshaciendo los efectos de las instrucciones posteriores.
3. **Parar la emisión si se detecta riesgo de excepción:** antes de ejecutar una instrucción que podría causar excepción (como una división), se detiene la emisión de nuevas instrucciones hasta que se confirma que no hay excepción.

En caso de múltiples excepciones simultáneas (por ejemplo, una excepción en la instrucción que está en EX y otra en la instrucción que está en MEM), el procesador gestiona **primero la más antigua** (la de la instrucción con menor número de secuencia, que es la que ha avanzado más en el pipeline).

2.11 Caso práctico: MIPS R4000 (relevante para el examen)

Nota de examen Este tema ha caído en examen

El procesador MIPS R4000 es el caso práctico más importante del Tema 2 desde el punto de vista del examen. Se deben conocer las diferencias con el MIPS de 5 etapas, las etapas adicionales y su efecto sobre las latencias y la gestión de saltos.

El **MIPS R4000** extiende el pipeline estándar de 5 etapas a **8 etapas**, descomponiendo el acceso a la caché (que en el MIPS básico se realiza en un solo ciclo) en múltiples etapas para permitir una frecuencia de reloj más alta. Las ocho etapas son:

Cuadro 2.6: Etapas del pipeline del MIPS R4000

Etapa	Nombre	Función
IF	Instruction Fetch (1ª mitad)	Envío del PC a caché de instrucciones
IS	Instruction Fetch (2ª mitad)	Completar lectura de instrucción de caché
RF	Register Fetch	Chequeo de etiqueta caché + decodificación + lectura registros + detección de conflictos
EX	Execute	Operación ALU + evaluación condición de salto + cálculo dir. salto
DF	Data Fetch (1ª mitad)	Inicio de lectura/escritura de datos en caché
DS	Data Fetch (2ª mitad)	Completar lectura/escritura de datos en caché
TC	Tag Check	Verificación de etiqueta caché para acierto/fallo
WB	Write Back	Escritura del resultado en banco de registros

2.11.1 Diferencias con el MIPS de 5 etapas

1. **Mayor latencia para instrucciones dependientes de load:** dado que los datos de memoria no están disponibles hasta el final de la etapa DS (etapa 6), una instrucción que depende de una carga necesita más ciclos de forwarding que en el MIPS de 5 etapas. Los caminos de adelantamiento van desde DS, TC y WB hacia EX.
2. **Mayor penalización por saltos:** los saltos condicionales se resuelven en la etapa EX (etapa 4). Entre la instrucción de salto (en IF) y la resolución del salto (en EX) hay 3 ciclos, lo que supone una penalización de 3 ciclos. Para mitigar esto, el R4000 usa:
 - **1 ranura de retardo** (siempre se ejecuta la instrucción inmediatamente después del salto).
 - **Predicción fija de salto no tomado:** si el salto se predice “no tomado” y la predicción falla, se anulan las instrucciones emitidas en los ciclos IS+2 e IS+3 (2 ciclos de penalización).

Para saltos incondicionales, la dirección se calcula en la etapa RF (etapa 3).

3. **Penalización de predicción incorrecta:** si el salto se toma pero se predijo “no tomado”, la instrucción en la ranura de retardo se ejecuta siempre (si es coherente) o se anula (si no lo es). Las dos instrucciones emitidas tras la ranura se anulan, produciendo una pérdida de **2 ciclos**.

BR	IF	IS	RF	EX	DF	DS	TC	WB
Slot		IF	IS	RF	EX	DF	DS	TC
IS+2			IF	IS	RF	EX	DF	DS
IS+3				IF	IS	RF	EX	DF
IS+4					IF	IS	RF	EX

Figura 2.5: Pipeline MIPS R4000: caso de salto no tomado con predicción correcta. No se pierde ningún ciclo.

2.12 Planificación de bucles y desenrollado

La planificación del compilador y el desenrollado de bucles son las técnicas de optimización de código más importantes para reducir el impacto de los riesgos de

BR	IF	IS	RF	EX	DF	DS	TC	WB
Slot		IF	IS	RF	EX	DF	DS	TC
IS+2			IF	IS	NOP	NOP	NOP	NOP
IS+3				IF	NOP	NOP	NOP	NOP
Destino Salto					IF	IS	RF	EX

Figura 2.6: Pipeline MIPS R4000: caso de predicción de salto incorrecta. Se anulan IS+2 e IS+3, perdiendo 2 ciclos efectivos (el delay slot IS+1 siempre se ejecuta si es coherente).

datos en los pipelines con operaciones multiciclo.

2.12.1 Planificación del compilador (*instruction scheduling*)

El **scheduling** de instrucciones consiste en reordenar las instrucciones del bucle para aprovechar las latencias entre instrucciones dependientes. La idea es que mientras una instrucción lenta (por ejemplo, una carga) espera a que el dato esté disponible, en ese tiempo se ejecuten instrucciones independientes que de otra forma estarían esperando.

La metodología es:

1. Identificar todas las dependencias RAW y sus latencias.
2. Construir un grafo de dependencias donde los nodos son instrucciones y los arcos representan dependencias con su peso (latencia en ciclos).
3. Reordenar las instrucciones siguiendo el camino crítico del grafo, intercalando instrucciones independientes para cubrir los huecos.

2.12.2 Desenrollado de bucles (*loop unrolling*)

El **desenrollado de bucles** (loop unrolling) con factor k consiste en replicar el cuerpo del bucle k veces, reduciendo el número de iteraciones a n/k y aumentando la cantidad de código disponible para el scheduler.

Ventajas del desenrollado:

- Expone más instrucciones independientes, permitiendo al scheduler cubrir mejor las latencias de las operaciones lentas.
- Reduce el overhead del propio bucle (instrucciones de actualización del contador, comparación y salto), que se amortiza entre k iteraciones.

- Permite usar más registros (los k conjuntos de variables necesitan k grupos de registros distintos para evitar dependencias entre iteraciones).

Desventajas:

- Aumenta el tamaño del código.
- Requiere que n sea divisible por k (o añadir código especial para el resto).
- Puede generar presión sobre el banco de registros (más registros necesarios).

2.13 Ejercicios resueltos con metodología de examen

Ejercicio resuelto 1 — Dependencias RAW, planificación y desenrollado de bucle FP

Enunciado. Procesador MIPS segmentado con banco de registros enteros (32 registros) y FP (16 registros de doble precisión: \$f0, \$f2, ..., \$f30). La instrucción `bnez` usa bifurcación retardada con 1 ranura de retardo. Latencias adicionales cuando hay dependencia RAW: `ldc1` → FP: 2 ciclos; `add.d` → FP: 4 ciclos; `mul.d` → FP: 6 ciclos.

Código original:

```

1 Bucle: ldc1    $f0, 0($t0)    # I1: carga elemento A[i]
2         ldc1    $f2, 0($t1)    # I2: carga elemento B[i]
3         add.d   $f4, $f0, $f2  # I3: A[i] + B[i]
4         mul.d   $f4, $f4, $f6  # I4: (A[i]+B[i]) * factor
5         sdc1    $f4, ($t2)     # I5: almacena resultado
6         addi    $t0, $t0, 8     # I6: actualiza puntero A
7         addi    $t1, $t1, 8     # I7: actualiza puntero B
8         subi    $t3, $t3, 1     # I8: decrementa contador
9         bnez    $t3, bucle     # I9: salto al inicio
10        addi    $t2, $t2, 8     # I10: delay slot: actualiza
                                puntero C

```

Parte 1: Dependencias RAW

Las dependencias RAW se identifican analizando qué registro escribe cada instrucción y qué instrucciones posteriores leen ese registro:

Cuadro 2.7: Dependencias RAW en el bucle original

Registro	Productora	Consumidora	Latencia adicional
\$f0	I1 (<code>ldc1</code>)	I3 (<code>add.d</code>)	2 ciclos
\$f2	I2 (<code>ldc1</code>)	I3 (<code>add.d</code>)	2 ciclos
\$f4	I3 (<code>add.d</code>)	I4 (<code>mul.d</code>)	4 ciclos
\$f4	I4 (<code>mul.d</code>)	I5 (<code>sdcl</code>)	6 ciclos
\$t3	I8 (<code>subi</code>)	I9 (<code>bnez</code>)	0 (enteros, forwarding)

Parte 2: Código con detenciones y recuento de ciclos


```

1 Bucle: ldc1    $f0, 0($t0)    # I1  ciclo 1
2         ldc1    $f2, 0($t1)    # I2  ciclo 2
3         <stall>                #      ciclo 3  (latencia ldc1->
4         add.d: 2 ciclos;
5         <stall>                #      ciclo 4  I2 termina ciclo
6         2, add.d necesita
7         #                      #      esperar hasta
8         #                      #      ciclo 5 -> 2 stalls)
9         add.d $f4, $f0, $f2    # I3  ciclo 5
10        <stall>                #      ciclo 6
11        <stall>                #      ciclo 7  (latencia add.d->
12        mul.d: 4 ciclos
13        <stall>                #      ciclo 8  add.d termina
14        ciclo 5, mul.d debe
15        <stall>                #      ciclo 9  esperar hasta
16        ciclo 10 -> 4 stalls)
17        mul.d $f4, $f4, $f6    # I4  ciclo 10
18        <stall> x6              #      ciclos 11-16 (latencia mul
19        .d->sdc1: 6 ciclos)
20        sdc1    $f4, ($t2)      # I5  ciclo 17
21        addi    $t0, $t0, 8      # I6  ciclo 18
22        addi    $t1, $t1, 8      # I7  ciclo 19
23        subi    $t3, $t3, 1      # I8  ciclo 20
24        bnez    $t3, bucle       # I9  ciclo 21
25        addi    $t2, $t2, 8      # I10 ciclo 22 (delay slot:
26        siempre ejecuta)

```

Listing 2.7: Bucle con stalls explícitos (22 ciclos por iteración)

Total: 22 ciclos por iteración.

El salto bnez en I9 con 1 ranura de retardo no genera parada por riesgo de control: la instrucción I10 en la ranura siempre se ejecuta y es independiente de la condición de salto.

Parte 3: Planificación del bucle (19 ciclos)

El objetivo es intercalar instrucciones independientes para cubrir las latencias. Las instrucciones I6, I7 e I8 (enteros, sin dependencias con FP) pueden moverse entre las instrucciones FP:

```

1 Bucle: ldc1    $f0, ($t0)      # I1  ciclo 1
2         ldc1    $f2, ($t1)      # I2  ciclo 2
3         addi    $t0, $t0, 8      # I6  ciclo 3  (cubre 1 stall de
4         ldc1->add.d)
5         addi    $t1, $t1, 8      # I7  ciclo 4  (cubre 2 stalls de
6         ldc1->add.d)
7         add.d    $f4, $f0, $f2    # I3  ciclo 5  (latencia
8         satisfecha: 2 ciclos util.)
9         subi    $t3, $t3, 1      # I8  ciclo 6  (cubre 1 stall de
10        add.d->mul.d)
11        <stall>                #      ciclo 7
12        <stall>                #      ciclo 8  (3 stalls

```

```

    restantes de 4)
9      <stall>                                #      ciclo 9
10     mul.d $f4, $f4, $f6 # I4 ciclo 10 (latencia
    satisfecha: 4 ciclos)
11     <stall> x6                                #      ciclos 11-16
12     sdc1 $f4, ($t2) # I5 ciclo 17
13     bnez $t3, bucle # I9 ciclo 18
14     addi $t2, $t2, 8 # I10 ciclo 19 (delay slot)

```

Listing 2.8: Bucle planificado (19 ciclos por iteración)

Total: 19 ciclos por iteración (ahorro de 3 ciclos frente al original).

Parte 4: Desenrollado con factor 4 (6,5 ciclos/iteración)

Se procesan 4 elementos en una sola iteración del bucle desenrollado. Cada elemento usa un juego de registros FP diferente para evitar dependencias entre iteraciones:

```

1 Bucle: ldc1 $f0, 0($t0) # carga A[i]
2         ldc1 $f2, 0($t1) # carga B[i]
3         ldc1 $f8, 8($t0) # carga A[i+1]
4         ldc1 $f10, 8($t1) # carga B[i+1]
5         ldc1 $f14, 16($t0) # carga A[i+2]
6         ldc1 $f16, 16($t1) # carga B[i+2]
7         ldc1 $f20, 24($t0) # carga A[i+3]
8         ldc1 $f22, 24($t1) # carga B[i+3]
9         add.d $f4, $f0, $f2 # A[i]+B[i] (latencia 2
    ciclos ya satisfecha)
10        add.d $f12, $f8, $f10 # A[i+1]+B[i+1]
11        add.d $f18, $f14, $f16 # A[i+2]+B[i+2]
12        add.d $f24, $f20, $f22 # A[i+3]+B[i+3]
13        <stall> # 1 stall (latencia add.d->
    mul.d = 4; 3 cubiertos)
14        mul.d $f4, $f4, $f6 # resultado[i]
15        mul.d $f12, $f12, $f6 # resultado[i+1]
16        mul.d $f18, $f18, $f6 # resultado[i+2]
17        mul.d $f24, $f24, $f6 # resultado[i+3]
18        addi $t0, $t0, 32 # avanza puntero A 4 elementos
    *8B = 32B
19        addi $t1, $t1, 32 # avanza puntero B
20        subi $t3, $t3, 4 # decrementa contador en 4
21        sdc1 $f4, 0($t2) # almacena resultado[i] (
    latencia mul.d->sdc1
22        sdc1 $f12, 8($t2) # almacena resultado[i+1] =
    6; las 4 mul.d + las
23        sdc1 $f18, 16($t2) # almacena resultado[i+2]
    addi/subi cubren 6)
24        sdc1 $f24, 24($t2) # almacena resultado[i+3]
25        bnez $t3, bucle # salto
26        addi $t2, $t2, 32 # delay slot: avanza puntero C

```

Listing 2.9: Bucle desenrollado x4 (26 ciclos para 4 iteraciones)

Total: 26 ciclos para 4 iteraciones, es decir, 6,5 ciclos por iteración. Speedup respecto al código original con stalls (22 ciclos):

$$S = \frac{22}{6,5} \approx 3,38$$

Speedup respecto al código planificado (19 ciclos):

$$S = \frac{19}{6,5} \approx 2,92$$

El beneficio del desenrollado proviene de: (a) exponer más instrucciones independientes que cubren las latencias, (b) reducir la fracción de tiempo dedicada al overhead del bucle (addi/subi/bnez) de 3/10 instrucciones a 3/26 instrucciones.

Ejercicio resuelto 2 — Diagrama de tiempos: sin forwarding vs. con forwarding

Enunciado. Bucle con las instrucciones:

```

1 Bucle: lw    $f0, 0($r1)
2         lw    $f2, 0($r2)
3         mul.f $f4, $f0, $f2
4         add.d $f6, $f6, $f4
5         addi  $r1, $r1, 4
6         addi  $r2, $r2, 4
7         sub   $r3, $r3, 1
8         bnez  $r3, bucle

```

Parte 1: Lista de dependencias RAW y WAR

Cuadro 2.8: Dependencias RAW y WAR en el bucle

Registro	Tipo	Productora	Consumidora
\$f0	RAW	I1 (lw)	I3 (mul.f)
\$f2	RAW	I2 (lw)	I3 (mul.f)
\$f4	RAW	I3 (mul.f)	I4 (add.d)
\$f6	RAW	I4 (add.d)	I4 (iteración siguiente)
\$r3	RAW	I7 (sub)	I8 (bnez)
\$r1	WAR	I1 (lw) lee	I5 (addi) escribe
\$r2	WAR	I2 (lw) lee	I6 (addi) escribe
\$r3	WAR	I8 (bnez) lee	I7 (sub) escribe

Nota: las WAR no generan paradas en el MIPS de 5 etapas (escrituras en etapa 5, lecturas en etapa 2).

Parte 2: Sin forwarding, saltos por vaciado de pipeline

Sin forwarding, cada instrucción dependiente debe esperar hasta que la productora complete WB. Las instrucciones lw tienen latencia 1 ciclo de acceso a memoria. Los saltos se gestionan vaciando el pipeline: la dirección se calcula

en ID pero la condición se resuelve en EX → se pierden 2 ciclos (instrucción en ID y otra en EX deben descartarse).

Para calcular los ciclos por iteración con N iteraciones:

- Sin forwarding, cada RAW entre instrucciones consecutivas cuesta 2 stalls.
- El salto cuesta 2 ciclos de penalización cada vez que se toma (todas las iteraciones excepto la última).
- Con N iteraciones: $T_{\text{sin fw}} = N \times (\text{ciclos_iter}) + \text{ciclos_arranque}$.

Parte 4: Con forwarding completo y predicción de salto tomado

Con forwarding, los riesgos RAW entre instrucciones ALU desaparecen o se reducen a 1 ciclo (load-use). Con predicción de salto tomado, las últimas instrucciones de la iteración siguiente comienzan a captarse inmediatamente, sin perder ciclos cuando el salto se toma. Solo hay penalización en la *última* iteración (cuando el salto no se toma pero se predijo tomado).

Comparación: el forwarding elimina la mayoría de los stalls por RAW, y la predicción de salto tomado elimina la penalización por salto en todas las iteraciones excepto la última. El speedup puede ser de $3\times$ o más dependiendo de las latencias específicas.

Ejercicio resuelto 3 — Riesgos estructurales forzados: bucle anidado con tabla de latencias

Enunciado. Máquina con riesgos estructurales forzados según tabla de latencias (el stall se produce *siempre* independientemente de si hay o no dependencia de datos). R1=0, R2=24, R3=16.

```

1 Loop1: LD      F2, 0(R1)      # carga elemento
2          ADDD   F2, F0, F2    # suma FP
3 Loop2: LD      F4, 0(R3)      # carga elemento matriz
4          MULTD  F4, F0, F4    # multiplicacion FP
5          DIVD   F10, F4, F0    # division FP (no segmentada)
6          ADDD   F12, F10, F4  # suma FP
7          ADDI   R1, R1, #8     # actualiza indice
8          SUB    R18, R2, R1    # calcula limite
9          BNZ    R18, Loop2     # salto bucle interior
10         SD     F2, 0(R3)      # almacena resultado
11         ADDI   R3, R3, #8     # actualiza puntero
12         SUB    R20, R2, R3    # calcula limite
13         BNZ    R20, Loop1     # salto bucle exterior

```

Metodología de solución:

Para resolver este ejercicio se deben seguir los siguientes pasos:

1. **Identificar la tabla de latencias:** cada tipo de instrucción tiene una latencia mínima asociada por riesgos estructurales que se produce siempre, independientemente de las dependencias. Con esta tabla se calcula el número de stalls que sigue a cada instrucción.

2. **Calcular ciclos del bucle interior** (Loop2) con 3 iteraciones ($R3 = 16$, procesa posiciones 16, 8, 0 de la matriz).
3. **Calcular ciclos del bucle exterior** (Loop1) que envuelve 3 iteraciones de Loop2.
4. **Aplicar el desenrollado** de Loop2 con factor 3 y recalcularlo. Al desenrollar, las instrucciones de loop overhead (ADDI, SUB, BNZ) solo aparecen una vez por cada 3 iteraciones del bucle original. Además, el scheduler puede aprovechar instrucciones independientes de las tres iteraciones para cubrir las latencias de DIVD y MULTD.
5. **Comparar:** el speedup del desenrollado proviene principalmente de reducir el overhead del bucle y de permitir al scheduler cubrir las largas latencias de DIVD con instrucciones útiles de otras iteraciones.

¡Atención!

En este ejercicio los stalls son **estructurales** (se producen siempre), no solo cuando hay dependencia de datos. Esto difiere del comportamiento normal del MIPS. Leer la tabla de latencias con atención antes de calcular los ciclos.

2.14 Resumen y conclusiones del Tema 2

Las ideas fundamentales que el alumno debe dominar para el examen son:

1. **Las 5 etapas del pipeline MIPS** (IF, ID, EX, MEM, WB) y qué hace cada instrucción en cada una de ellas. Incluso las instrucciones que no usan una etapa (por ejemplo, add en MEM) deben atravesarla para mantener la sincronía del cauce.
2. **Los tres tipos de riesgos:** estructurales, de datos (RAW, WAR, WAW) y de control. Solo RAW puede ocurrir en el MIPS de 5 etapas para enteros; WAW también puede ocurrir en FP multiciclo.
3. **Forwarding:** las condiciones exactas que lo activan, los dos caminos de bypass (desde EX/MEM y desde MEM/WB hacia las entradas de la ALU), y el caso que no puede resolver (load-use).
4. **Control de interbloqueo:** la condición de activación (load en EX + instrucción ALU/salto/store en ID con el mismo registro destino/fuente) y el efecto (1 ciclo de parada + burbuja en EX).
5. **Riesgos de control:** la diferencia entre congelación, predicción fija (no tomado / tomado), bifurcación retardada y predicción dinámica. Las tres estrategias para rellenar el delay slot.
6. **Predictores dinámicos:** 1 bit, 2 bits, información local, información global y predictor combinado. Saber dibujar la máquina de estados del predictor de 2 bits.

7. **Operaciones FP multiciclo:** latencias adicionales (ldc1: 2, add.d: 4, mul.d: 6 para el ejercicio del examen), y los riesgos WAW y estructurales adicionales que introduce.
8. **Planificación y desenrollado:** metodología para reducir stalls reordenando instrucciones y usando múltiples registros en el desenrollado.
9. **MIPS R4000:** 8 etapas, resolución de saltos en EX (4 ciclos antes del destino), 1 delay slot + predicción no tomado, penalización de 2 ciclos por predicción incorrecta.

Fórmulas y reglas rápidas para el examen

CPI efectivo con riesgos:

$$CPI_{ef} = 1 + \text{stalls/instrucción}$$

Ciclos de stall por riesgo RAW (sin forwarding): número de ciclos que la instrucción productora tiene que avanzar desde la etapa actual hasta WB, menos 1 (por el read-write en mismo ciclo).

Detección de interbloqueo (load-use):

$$EX.lw \wedge (EX.rd = ID.rs \vee EX.rd = ID.rt) \Rightarrow \text{stall 1 ciclo}$$

Speedup del desenrollado con factor k :

$$S = \frac{\text{ciclos_original} \times k}{\text{ciclos_desenrollado}}$$

Penalización por saltos:

- Congelación / predicción incorrecta en MIPS 5 etapas: 1 ciclo
- Predicción incorrecta en MIPS R4000: 2 ciclos
- Con delay slot (1 ranura): penalización reducida en 1 ciclo

Latencias FP para el ejercicio tipo (referencia del examen):

Instrucción productora	Stalls adicionales
ldc1 → operación FP	2
add.d → operación FP	4
mul.d → operación FP	6